

```
<h1 align="center"><img alt="Stock Price Analyzer logo" data-bbox="225 51 242 64"/> Stock Price Analyzer</h1>

<p align="center">
  <b>C++ (DSA) + Machine Learning based stock analysis and prediction system</b>
</p>

<p align="center">
  
  
  
  
</p>
```

🚀 Overview

This project combines **Data Structures & Algorithms (C++)** with **Machine Learning (Python)** to analyze stock price data, evaluate trading strategies, and predict future prices.

It bridges **core computer science concepts** with **real-world financial applications**, making it highly relevant for both **software engineering** and **finance roles**.

⚙️ Features

- ### 💎 DSA-Based Analysis (C++)
- Heap-based price tracking for efficient max/min queries
 - Stack-based trend analysis
 - Optimized profit calculation strategies

- ### 💎 Machine Learning Models
- Linear Regression
 - Random Forest ✅ ***(Best Model)***
 - XGBoost

- ### 💎 Performance Evaluation
- Metric: **Mean Absolute Error (MAE)**
 - Model comparison and benchmarking

- ### 💎 Interactive Dashboard
- Visual comparison of models
 - Actual vs Predicted price plots
 - Clean and intuitive UI

📊 Results

Model	MAE ↓
Linear Regression	2.72
Random Forest	**1.19** ⭐
XGBoost	1.78

👉 **Best Model: Random Forest (lowest error, better generalization)**

👉 Derived features from raw stock data to capture trends, momentum, and trading opportunities

👉 Includes stock span, running max/min prices, and profit potential indicators

```

```

```

```

👉 Model comparison based on MAE (lower is better)

👉 Visualization of prediction error – highlights model accuracy and consistency

```

```

```

```

🔧 Tech Stack

Languages:

C++
Python

Libraries & Tools:

pandas, numpy
scikit-learn
XGBoost
matplotlib
Streamlit

📁 Project Structure

```

```
Stock-price-analyzer/
├── data/
├── include/ # Header files (Heap, Stack, Profit logic)
├── src/ # C++ implementations
├── dashboard.py # Visualization UI
├── download_data.py
├── train_linear_model.py
├── train_random_forest.py
├── train_xgboost.py
└── main.exe
```

```

⚙️ Installation

```
```bash
git clone https://github.com/dhairya2006-del/Stock-price-analyzer.git
cd Stock-price-analyzer
pip install -r requirements.txt
```
```

▶ Usage

Run ML Pipeline

```
```bash
python download_data.py
python train_linear_model.py
python train_random_forest.py
python train_xgboost.py
```
```

Run Dashboard

```
```bash
streamlit run dashboard.py
```
```

Run C++ Code

```
```bash
g++ src/main.cpp src/heap_analysis.cpp src/stack_analysis.cpp src/profit_analysis.cpp -o main
./main
```
```

🌸 How It Works

1. Collect stock price data
2. Generate features for prediction
3. Apply DSA techniques:
 - * Heap → efficient price tracking
 - * Stack → trend detection
 - * Profit algorithms → trading insights
4. Train ML models
5. Evaluate using MAE
6. Visualize results in dashboard

🌟 Future Improvements

- * LSTM / Deep Learning models
- * Real-time API integration (Yahoo Finance)
- * Backtesting trading strategies
- * Portfolio optimization
- * Deployment (AWS / Streamlit Cloud)

💡 Key Learnings

- * Applied DSA concepts to financial data
- * Built an end-to-end ML pipeline
- * Compared multiple models effectively
- * Learned importance of ensemble methods (Random Forest)